BITS PILANI, DUBAI CAMPUS
DUBAI INTERNATIONAL ACADEMIC CITY, DUBAI
SECOND SEMESTER 2025 - 2026

Data Structures & Algorithms (CS F211)

```c
/*
Name: Kirtika
Roll No: 2023A7PS0219U
Course: CS F211 – Data Structures & Algorithms
Lab Exercise 8: Hash Table with Separate Chaining
*/

#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_SIZE 7
#define MAX_KEY_LEN 100

typedef struct Node {
    char key[MAX_KEY_LEN];
    struct Node *next;
} Node;

static int hash_value(const char *key) {
    int alpha_sum = 0;
    int digit_sum = 0;
    int symbol_sum = 0;

    for (size_t i = 0; key[i] != '\0'; i++) {
        unsigned char ch = (unsigned char)key[i];
        if (isalpha(ch)) {
            alpha_sum += ch;
        } else if (isdigit(ch)) {
            digit_sum += ch;
        } else {
            symbol_sum += ch;
        }
    }

    return (alpha_sum + 3 * digit_sum + 5 * symbol_sum + 7) % MAX_SIZE;
}

static Node *create_node(const char *key) {
    Node *node = (Node *)malloc(sizeof(Node));
    if (node == NULL) {
        return NULL;
    }
```

```c
    strncpy(node->key, key, MAX_KEY_LEN - 1);
    node->key[MAX_KEY_LEN - 1] = '\0';
    node->next = NULL;
    return node;
}

static int insert_key(Node *table[], const char *key) {
    int index = hash_value(key);
    Node *node = create_node(key);

    if (node == NULL) {
        return 0;
    }

    node->next = table[index];
    table[index] = node;
    return 1;
}

static int search_key(Node *table[], const char *key, int *out_index) {
    int index = hash_value(key);
    Node *current = table[index];

    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            if (out_index != NULL) {
                *out_index = index;
            }
            return 1;
        }
        current = current->next;
    }

    return 0;
}

static int delete_key(Node *table[], const char *key) {
    int index = hash_value(key);
    Node *current = table[index];
    Node *previous = NULL;

    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            if (previous == NULL) {
                table[index] = current->next;
            } else {
                previous->next = current->next;
            }
            free(current);
```

```
            return 1;
        }

        previous = current;
        current = current->next;
    }

    return 0;
}

static void display_table(Node *table[]) {
    printf("Hash table\n");
    for (int i = 0; i < MAX_SIZE; i++) {
        printf("[%d]:", i);
        Node *current = table[i];
        while (current != NULL) {
            printf(" -> %s", current->key);
            current = current->next;
        }
        printf("\n");
    }
}

static void free_table(Node *table[]) {
    for (int i = 0; i < MAX_SIZE; i++) {
        Node *current = table[i];
        while (current != NULL) {
            Node *next = current->next;
            free(current);
            current = next;
        }
        table[i] = NULL;
    }
}

int main(void) {
    Node *table[MAX_SIZE] = {NULL};

    const char *inputs[] = {
        "M2y", "N3x", "F4w", "O5v", "D*2u", "A2t", "K5#y", "M6z", "N7a", "Y3w",
        "b2Y", "e3X", "f4W", "c5V", "d2U", "a2T", "J5Y", "m6%Z", "n7A", "y3W"
    };
    int count = (int)(sizeof(inputs) / sizeof(inputs[0]));

    printf("Hash values for given input strings:\n");
    for (int i = 0; i < count; i++) {
        int hv = hash_value(inputs[i]);
        printf("Hash value for %s = %d\n", inputs[i], hv);
        if (!insert_key(table, inputs[i])) {
            fprintf(stderr, "Insertion failed for key: %s\n", inputs[i]);
```

```
            free_table(table);
            return 1;
        }
    }

    printf("\n");
    display_table(table);

    const char *delete_demo = "b2Y";
    printf("\nDelete: %s\n", delete_demo);
    if (!delete_key(table, delete_demo)) {
        printf("Key %s not found for deletion.\n", delete_demo);
    }
    display_table(table);

    const char *search_demo_1 = "a2T";
    int index;
    printf("\nSearch: %s\n", search_demo_1);
    if (search_key(table, search_demo_1, &index)) {
        printf("Key found at index %d\n", index);
    } else {
        printf("Key not found\n");
    }

    const char *search_demo_2 = "wew423";
    printf("\nSearch: %s\n", search_demo_2);
    if (search_key(table, search_demo_2, &index)) {
        printf("Key found at index %d\n", index);
    } else {
        printf("Key not found\n");
    }

    int choice;
    char key[MAX_KEY_LEN];
    do {
        printf("\nMenu:\n");
        printf("1. Insert key\n");
        printf("2. Search key\n");
        printf("3. Delete key\n");
        printf("4. Display hash table\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid choice input.\n");
            break;
        }

        if (choice == 1) {
            printf("Enter key to insert: ");
```

```c
            if (scanf("%99s", key) != 1) {
                printf("Invalid key input.\n");
                continue;
            }
            if (insert_key(table, key)) {
                printf("Inserted. Hash value = %d\n", hash_value(key));
            } else {
                printf("Insertion failed due to memory allocation error.\n");
            }
        } else if (choice == 2) {
            printf("Enter key to search: ");
            if (scanf("%99s", key) != 1) {
                printf("Invalid key input.\n");
                continue;
            }
            if (search_key(table, key, &index)) {
                printf("Key found at index %d\n", index);
            } else {
                printf("Key not found\n");
            }
        } else if (choice == 3) {
            printf("Enter key to delete: ");
            if (scanf("%99s", key) != 1) {
                printf("Invalid key input.\n");
                continue;
            }
            if (delete_key(table, key)) {
                printf("Key deleted.\n");
            } else {
                printf("Key not found.\n");
            }
        } else if (choice == 4) {
            display_table(table);
        } else if (choice == 5) {
            printf("Exiting program.\n");
        } else {
            printf("Please choose between 1 and 5.\n");
        }
    } while (choice != 5);

    free_table(table);
    return 0;
}
```

OUT PUT

```
1
abc123
2
abc123 …
manavaryasingh@MacBook-Pro-5 kritika-dsa %  cd "/Users/manavaryasingh/kritika-dsa" && gcc "week 8/ex8_hash_table_chaining.c" -o "week 8/ex8" && "week 8/ex8" << 'EOF'
1
abc123
2
abc123
3
abc123
4
5
EOF
Hash values for given input strings:
Hash value for M2y = 5
Hash value for N3x = 1
Hash value for F4w = 2
Hash value for O5v = 6
Hash value for D*2u = 6
Hash value for A2t = 2
Hash value for K5#y = 5
Hash value for M6z = 4
Hash value for N7a = 4
Hash value for Y3w = 4
Hash value for b2Y = 1
Hash value for e3X = 6
Hash value for f4W = 2
Hash value for c5V = 1
Hash value for d2U = 6
Hash value for a2T = 2
Hash value for J5Y = 0
Hash value for m6%Z = 0
Hash value for n7A = 4
Hash value for y3W = 4

Hash table
[0]: -> m6%Z -> J5Y
[1]: -> c5V -> b2Y -> N3x
[2]: -> a2T -> f4W -> A2t -> F4w
[3]:
[4]: -> y3W -> n7A -> Y3w -> N7a -> M6z
[5]: -> K5#y -> M2y
[6]: -> d2U -> e3X -> D*2u -> O5v

Delete: b2Y
Hash table
[0]: -> m6%Z -> J5Y
[1]: -> c5V -> N3x
[2]: -> a2T -> f4W -> A2t -> F4w
[3]:
[4]: -> y3W -> n7A -> Y3w -> N7a -> M6z
[5]: -> K5#y -> M2y
[6]: -> d2U -> e3X -> D*2u -> O5v

Search: a2T
Key found at index 2

Search: wew423
Key not found
```

```
manavaryasingh@MacBook-Pro-5 kritika-dsa %  cd "/Users/manavaryasingh/kritika-dsa" && gcc "week 8/ex8_hash_table_chaining.c" -o "week 8/ex8" && "week 8/ex8" << 'EOF'
1
abc123
2
abc123 …
[5]: -> K5#y -> M2y
[6]: -> d2U -> e3X -> D*2u -> O5v

Search: a2T
Key found at index 2

Search: wew423
Key not found

Menu:
1. Insert key
2. Search key
3. Delete key
4. Display hash table
5. Exit
Enter your choice: Enter key to insert: Inserted. Hash value = 2

Menu:
1. Insert key
2. Search key
3. Delete key
4. Display hash table
5. Exit
Enter your choice: Enter key to search: Key found at index 2

Menu:
1. Insert key
2. Search key
3. Delete key
4. Display hash table
5. Exit
Enter your choice: Enter key to delete: Key deleted.

Menu:
1. Insert key
2. Search key
3. Delete key
4. Display hash table
5. Exit
Enter your choice: Hash table
[0]: -> m6%Z -> J5Y
[1]: -> c5V -> N3x
[2]: -> a2T -> f4W -> A2t -> F4w
[3]:
[4]: -> y3W -> n7A -> Y3w -> N7a -> M6z
[5]: -> K5#y -> M2y
[6]: -> d2U -> e3X -> D*2u -> O5v

Menu:
1. Insert key
2. Search key
3. Delete key
4. Display hash table
5. Exit
Enter your choice: Exiting program.
manavaryasingh@MacBook-Pro-5 kritika-dsa %
```